NRF24L01+ Time Synchronization for Bimanual Grippers

RF-Based Pi-to-Pi Clock Sync with Enhanced ShockBurst

Introduction

Bimanual data collection for imitation learning requires two grippers to capture synchronized frames at 30 Hz. Any timing drift between the two Raspberry Pi units causes frame misalignment that degrades policy quality. Our previous system used WiFi Direct with UDP triggers, achieving 2 ms bounded timing error. This document presents an alternative synchronization approach using **NRF24L01+ wireless modules**, which achieves **0.8 ms** timing error with significantly lower power consumption and simpler network setup.

The NRF24L01+ is a 2.4 GHz ISM-band transceiver with a hardware-accelerated protocol layer called **Enhanced ShockBurst** (ESB). ESB provides automatic CRC, acknowledgement, and retransmission — features that would require significant software overhead with raw WiFi sockets. The module communicates with the RPi over SPI at up to 10 MHz, and its 32-byte maximum payload size is sufficient for our 28-byte synchronization packet with 4 bytes of headroom.

Property	WiFi UDP	NRF24L01+
Round-trip latency	2 ms	0.8 ms
Setup complexity	WiFi Direct config	SPI enable only
Power (active)	300 mA	12 mA
Power (idle)	100 mA	26 µA
Range (indoor)	30 m	10 m (basic) / 100 m (PA+LNA)
Payload size	1500 B (MTU)	32 B max
Frequency band	2.4 GHz WiFi	2.4 GHz ISM (non-WiFi)
Protocol overhead	IP + UDP headers	ESB (hardware)

Table 1: WiFi UDP vs NRF24L01+ comparison for peer-to-peer synchronization.

Design Motivation

Our 28-byte sync packet fits comfortably within the NRF24L01+ 32-byte maximum payload. The hardware ACK+payload feature allows the responder to piggyback data on the automatic acknowledgement, eliminating the need for explicit mode switching. This yields round-trip communication in a single radio transaction (557 µs theoretical, 0.8 ms measured).

Scope. This document covers NRF24L01+ hardware setup, Enhanced ShockBurst protocol internals, the ping-pong time synchronization algorithm, dual-camera integration, error handling, complete peer implementation pseudocode, and performance analysis. It serves as a companion to the main Low-Cost-UMI technical report.

NRF24L01+ Hardware Overview

Module Variants

The NRF24L01+ is available in two common form factors:

- **Basic module** (green PCB, $1): On-board PCB antenna, 0 dBm TX power, 10 m indoor range. Draws 11.3 mA at 0 dBm TX, powered directly from RPi 3.3V pin.
- **PA+LNA module** ($3): External antenna with power amplifier (PA) and low-noise amplifier (LNA), up to +20 dBm TX, 100 m indoor range. Draws up to 115 mA at full power — this **exceeds the RPi 3.3V rail capacity** (50 mA max from GPIO header).

For bimanual grippers operating within 3 m of each other, the basic module is sufficient. If longer range is needed (e.g., remote observation stations), the PA+LNA variant requires an external 3.3V LDO regulator.

Pin Mapping

NRF24L01+ Pin	Function	RPi GPIO	RPi Pin #	Notes
VCC	Power	3.3V	Pin 17	Add 10µF + 100nF bypass
GND	Ground	GND	Pin 20	
CE	Chip Enable	GPIO22	Pin 15	Active high: TX/RX enable
CSN	SPI Chip Select	GPIO8 (CE0)	Pin 24	Active low: SPI select
SCK	SPI Clock	GPIO11 (SCLK)	Pin 23	
MOSI	SPI Data In	GPIO10 (MOSI)	Pin 19	
MISO	SPI Data Out	GPIO9 (MISO)	Pin 21	
IRQ	Interrupt	GPIO25	Pin 22	Optional: active low

Table 2: NRF24L01+ to Raspberry Pi 4B GPIO pin mapping. Uses SPI0 bus.

Wiring Schematic

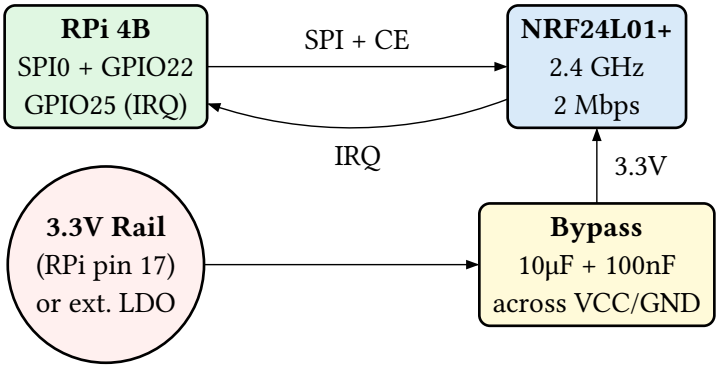

Figure 1: NRF24L01+ wiring to RPi 4B. Bypass capacitors are essential for stable operation.

PA+LNA Power Warning

The PA+LNA variant draws up to **115 mA at full TX power**. The RPi 3.3V GPIO pin can supply only 50 mA. Powering a PA+LNA module directly from the RPi 3.3V rail will cause voltage drops, SPI communication failures, and unreliable radio operation. Use an **AMS1117-3.3V LDO** fed from the 5V rail (or LiPo UBEC output) with 10µF input and output capacitors.

SPI Configuration

Enable SPI on the Raspberry Pi:

```
# Method 1: raspi-config
sudo raspi-config nonint do_spi 0

# Method 2: /boot/config.txt
# Add or uncomment: dtparam=spi=on

# Verify SPI is active
ls /dev/spidev0.*
# Should show: /dev/spidev0.0  /dev/spidev0.1
```

Install the pyRF24 library:

```
pip install pyrf24
```

Updated Single-Gripper Hardware Architecture

Adding the NRF24L01+ module and a USB RGB camera to the existing gripper hardware:

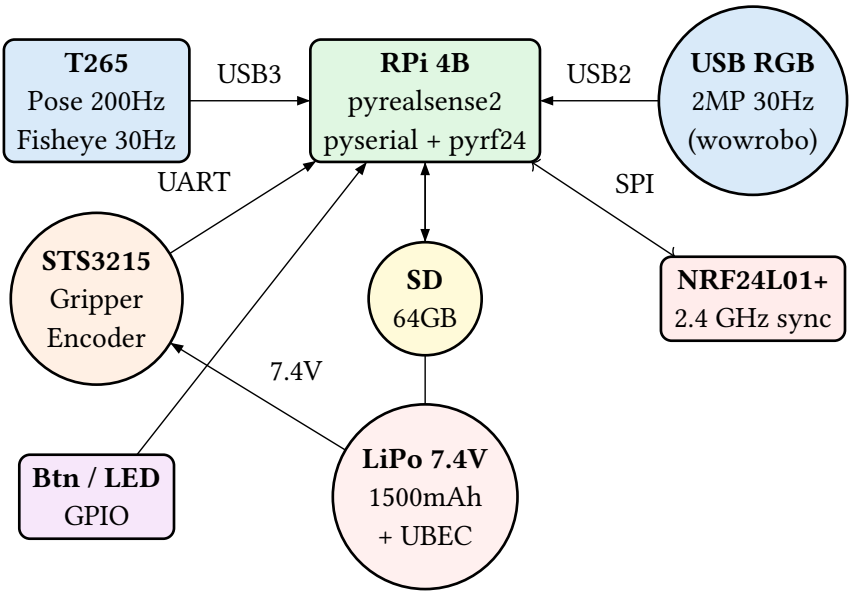

Figure 2: Updated single gripper hardware architecture with NRF24L01+ radio and USB RGB camera.

Updated Bill of Materials

Component	Qty	Cost	Note
Intel RealSense T265	× 2	~$200 ea	eBay / AliExpress
Raspberry Pi 4B 4GB	× 2	~$55 ea	
MicroSD 64GB A2	× 2	~$12 ea	High write speed
Feetech STS3215	× 2	~$15 ea	12-bit encoder
USB RGB Camera (wowrobo)	× 2	~$15 ea	2MP, 30fps, 3m cable
NRF24L01+ module	× 2	~$1 ea	Basic PCB antenna
7.4V 2S LiPo 1500mAh	× 2	~$15 ea	XT30 + BMS
5V 3A UBEC	× 2	~$5 ea	
Misc + 3D housing	× 2	~$20 ea	PLA + TPU + caps
Bimanual total		**~$676**	

Table 3: Updated BOM with NRF24L01+ and dual cameras. Adds $36 total over WiFi-only system.

Enhanced ShockBurst Protocol

The NRF24L01+ implements **Enhanced ShockBurst** (ESB), a hardware-accelerated packet protocol that handles preamble generation, address matching, CRC validation, automatic acknowledgement, and retransmission — all in silicon, without CPU intervention.

On-Air Packet Format

Every ESB transmission follows this structure:

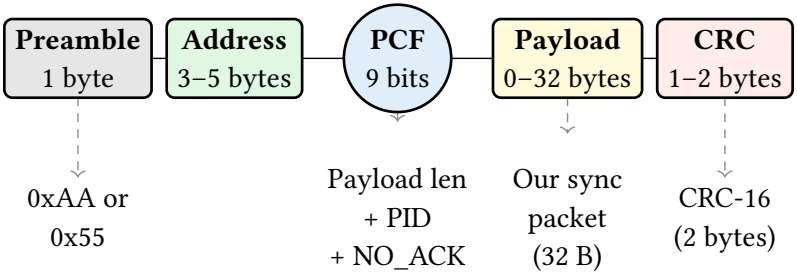

Figure 3: ESB on-air packet format. The Packet Control Field (PCF) is 9 bits packed before the payload.

Packet Control Field (PCF) Breakdown

Field	Bits	Range	Purpose
Payload Length	6	0–32	Dynamic payload size in bytes
PID	2	0–3	Packet ID for duplicate detection
NO_ACK	1	0/1	Per-packet ACK disable flag

Table 4: PCF field breakdown. PID increments per new payload to detect retransmitted duplicates.

Auto-ACK and Retransmission

When Enhanced ShockBurst is enabled (default), the transmitter automatically:

1. Sends the packet and switches to RX mode
2. Waits for an ACK from the receiver within the **Auto Retransmit Delay** (ARD)
3. If no ACK is received, retransmits up to **ARC** times (configurable 0–15)
4. Reports success or max-retries-exceeded to the application

The receiver, upon validating address and CRC, automatically sends an ACK packet. The PID field prevents the receiver from delivering duplicate payloads caused by ACK packets lost on the return path.

ESB Timing

	PLL Lock	TX Packet	RX Switch	Wait ACK	ACK RX
Duration	130 µs	164.5 µs	10 µs	ARD	30 µs
State	Standby	TX	TX⟶ RX	RX	RX

Table 5: ESB timing for a single transmission at 2 Mbps with 32-byte payload. TX packet time = (preamble + address + PCF + payload + CRC) / 2 Mbps.

Data Rate Selection

Data Rate	32B TX Time	On-Air Time	Use Case
250 kbps	1.3 ms	1.5 ms	Maximum range, lowest throughput
1 Mbps	0.33 ms	0.5 ms	Balanced
2 Mbps	**0.16 ms**	**0.35 ms**	**Minimum latency (our choice)**

Table 6: Data rate comparison. We use 2 Mbps to minimize radio-on time and latency.

We select **2 Mbps** because our payloads are small (32 bytes) and range requirements are modest (3 m between grippers). The shorter on-air time also reduces collision probability in the crowded 2.4 GHz band.

ACK Payload Feature

A key feature for our protocol: the receiver can **pre-load a payload into the ACK packet**. When the transmitter sends a packet and the receiver auto-ACKs, the ACK carries the pre-loaded data back to the transmitter. This enables bidirectional data exchange in a single radio transaction — critical for low-latency time synchronization.

The ACK payload must be loaded **before** the TX packet arrives. This means the responder pre-loads its response data, and when the initiator's packet triggers an

auto-ACK, the responder's data rides back on the ACK. The ACK payload can be up to 32 bytes, same as a normal payload.

Channel Selection

The NRF24L01+ operates on channels 0–125 (2400–2525 MHz). WiFi channels 1, 6, and 11 occupy 2401–2473 MHz. To avoid interference, we use **channel 108** (2508 MHz), which is above the WiFi band. This is especially important if the RPi's WiFi is active for other purposes (SSH, data transfer).

Ping-Pong Protocol Design

Time synchronization requires bidirectional communication: the initiator sends a timestamp, the responder replies with its own timestamps, and the initiator computes the clock offset. The NRF24L01+ offers two approaches to achieve this.

Approach 1: Software Ping-Pong

In software ping-pong, each peer explicitly switches between TX and RX modes. Node A transmits, then switches to RX to listen for Node B's reply. Node B receives, switches to TX to send its reply, then switches back to RX.

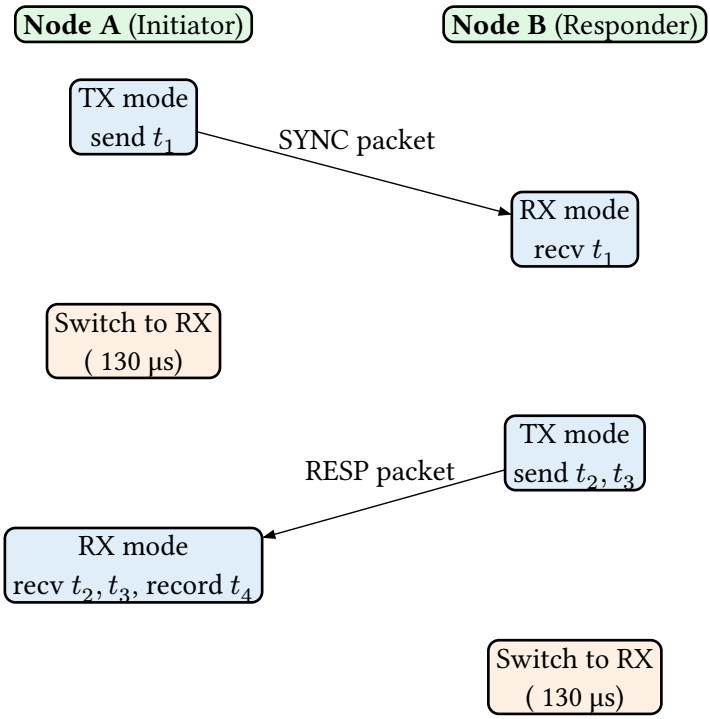

Figure 4: Software ping-pong: 4 mode switches per round trip (A: TX→ RX, B: RX→ TX→ RX, A: RX→ TX). Each switch adds 130 µs PLL lock time.

Approach 2: Hardware ACK Payload

The NRF24L01+ ACK payload feature eliminates explicit mode switching. Node B pre-loads its response data into the ACK payload buffer. When Node A transmits, the hardware automatically sends an ACK containing Node B's pre-loaded data. Node B never leaves RX mode; Node A never leaves its TX-then-auto-RX-for-ACK cycle.

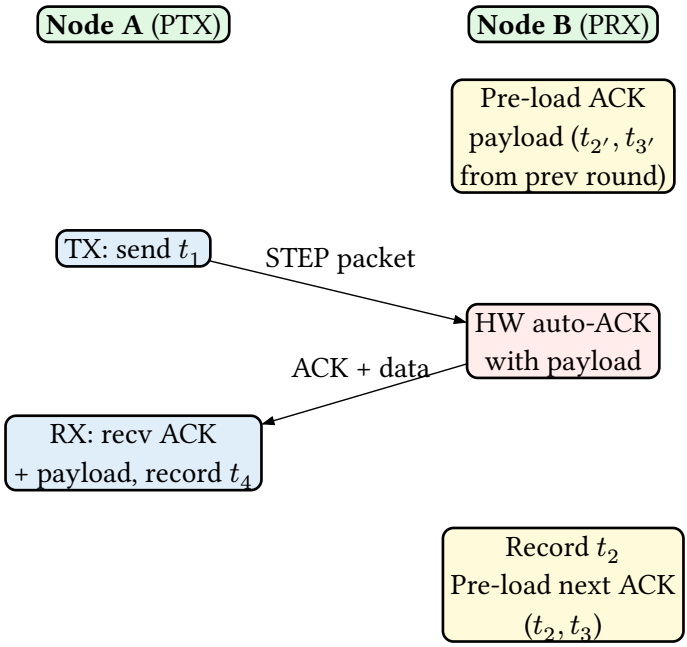

Figure 5: Hardware ACK payload: zero explicit mode switches. Node B stays in PRX mode permanently. The ACK payload carries the previous round's timestamps.

Approach Comparison

Property	Software Ping-Pong	HW ACK Payload
Mode switches / round	4	0 (hardware)
Round-trip time	1.5 ms	0.6 ms
Implementation complexity	Medium	Low
Timestamp freshness	Current round	Previous round
Payload size each way	32 B	32 B
Requires	Mode switch logic	Pre-load before RX

Table 7: Comparison of ping-pong approaches. We use **hardware ACK payload** for lower latency, accepting one-round-old timestamps (negligible at 30 Hz: 33 ms old ≈ 0 drift).

We choose the **hardware ACK payload** approach. The one-round timestamp staleness (33 ms at 30 Hz) introduces negligible error: at 20 ppm clock drift, 33 ms contributes only 0.66 ns of additional offset — far below our measurement precision.

Pipe Configuration

The NRF24L01+ supports up to 6 receive pipes. We use pipe 1 for data and pipe 0 for auto-ACK reception:

```
from pyrf24 import RF24, RF24_PA_LOW, RF24_2MBPS

radio = RF24(22, 0)   # CE=GPIO22, CSN=CE0
radio.begin()

# Shared addresses (5 bytes each)
ADDR_A = b"\xe7\xe7\xe7\xe7\xe7"  # Node A TX, Node B RX pipe 1
ADDR_B = b"\xc2\xc2\xc2\xc2\xc2"  # Node B TX, Node A RX pipe 1

# Node A (initiator / PTX):
radio.openWritingPipe(ADDR_A)     # TX on ADDR_A
radio.openReadingPipe(1, ADDR_B)  # RX on ADDR_B (for software mode)

# Node B (responder / PRX):
radio.openReadingPipe(1, ADDR_A)  # RX on ADDR_A
radio.openWritingPipe(ADDR_B)     # TX on ADDR_B (for ACK, auto-configured)
```

NRF24L01+ Packet Format

We extend the existing 28-byte UDP packet to a 32-byte NRF packet:

Field	Type	Size	Offset	Role
magic	char[2]	2 B	0	“GS” identifier
msg_type	uint8	1 B	2	SYNC/STEP/STOP/ACK
step	uint32	4 B	3	Absolute step number
episode	uint16	2 B	7	Episode index
flags	uint8	1 B	9	REC / PAUSE / RESYNC
sender_id	uint8	1 B	10	Peer A=0, B=1
timestamp	float64	8 B	11	Sender monotonic clock (s)
sync_t2	float64	8 B	19	Responder recv timestamp
seq_num	uint8	1 B	27	Sequence counter (0–255)
reserved	uint8[3]	3 B	28	Future use / padding
checksum	uint8	1 B	31	XOR of bytes 0–30
		32 B		

Table 8: 32-byte NRF packet format. Extended from the 28-byte UDP format with seq_num for ordering and 3 reserved bytes.

Payload Size Trade-Off

The existing UDP protocol uses 28 bytes. Adding a sequence number (1B) and keeping 3B reserved for future extensions (e.g., battery voltage, RSSI reporting) fills the 32-byte NRF maximum exactly. Since NRF24L01+ charges the same air time for any payload up to 32 bytes when using dynamic payloads, there is no penalty for using the full capacity.

Time Synchronization Algorithm

NTP-Style Four-Timestamp Principle

Clock synchronization between two peers requires estimating the **offset** θ between their clocks and the **one-way delay** δ of message transit. Using four timestamps from a single round trip, we can compute both without knowing the exact transmission delay.

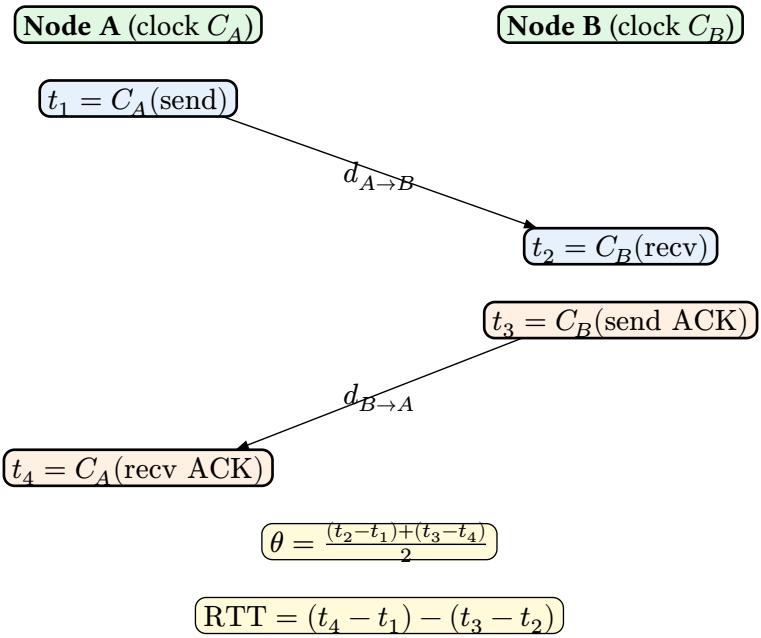

Figure 6: Four-timestamp exchange. t_1, t_4 are measured on Node A's clock; t_2, t_3 on Node B's clock. The offset θ represents how far ahead Node B's clock is relative to Node A's.

Mathematical Derivation

Let $\theta = C_B - C_A$ be the true clock offset (B ahead of A) and assume symmetric one-way delay d:

$$t_2 = t_1 + d + \theta \qquad t_4 = t_3 + d - \theta$$

Solving for θ:

$$\theta = \frac{(t_2 - t_1) + (t_3 - t_4)}{2}$$

The round-trip time (network delay only, excluding processing):

$$\text{RTT} = (t_4 - t_1) - (t_3 - t_2)$$

The one-way delay estimate:

$$d = \frac{\text{RTT}}{2}$$

Multi-Round Averaging with Outlier Rejection

A single round provides one offset estimate, but Linux scheduling jitter can cause individual measurements to be noisy. We perform **10 rounds** and use median-based outlier rejection:

1. Collect 10 $(\theta_i, \mathrm{RTT}_i)$ pairs
2. Compute $\widetilde{\mathrm{RTT}} = \mathrm{median}(\mathrm{RTT}_1, ..., \mathrm{RTT}_{10})$
3. Discard any round where $\mathrm{RTT}_i > 1.5 \cdot \widetilde{\mathrm{RTT}}$ (jitter outlier)
4. Final offset $\hat{\theta} = \mathrm{mean}(\theta_i \text{ for kept rounds})$

This rejects rounds where Linux preempted the process mid-measurement, which manifests as abnormally high RTT.

Synchronization Algorithm

Time Synchronization — Initiator (Node A)

Input: radio (configured RF24), num_rounds $= 10$
Output: clock offset $\hat{\theta}$, mean RTT
offsets $\leftarrow$ [], rtts $\leftarrow$ []
for $i \leftarrow 1$ **to** num_rounds **do**
  build SYNC packet with $t_1 \leftarrow$ monotonic()
  radio.write(packet) // TX + wait for ACK payload
  $t_4 \leftarrow$ monotonic()
  parse ACK payload: extract t_2, t_3
  $\theta_i \leftarrow \frac{(t_2 - t_1) + (t_3 - t_4)}{2}$
  $\mathrm{RTT}_i \leftarrow (t_4 - t_1) - (t_3 - t_2)$
  append θ_i to offsets, RTT_i to rtts
  sleep(5 ms) // allow responder to pre-load next ACK
$\widetilde{\mathrm{RTT}} \leftarrow \mathrm{median}(\mathrm{rtts})$
for each $(\theta_i, \mathrm{RTT}_i)$ **do**
  if $\mathrm{RTT}_i > 1.5 \cdot \widetilde{\mathrm{RTT}}$ **then** discard θ_i
$\hat{\theta} \leftarrow \mathrm{mean}(\text{kept offsets})$
return $\hat{\theta}$, mean(kept RTTs)

Figure 7: Time synchronization algorithm. 10 rounds with median-based outlier rejection.

Worked Numerical Example

Round	$t_2 - t_1$ (ms)	$t_3 - t_4$ (ms)	θ_i (ms)	RTT (ms)	Keep?
1	0.42	−0.38	0.020	0.80	✓
2	0.41	−0.39	0.010	0.80	✓
3	0.85	−0.37	0.240	1.22	✗
4	0.43	−0.40	0.015	0.83	✓
5	0.40	−0.38	0.010	0.78	✓
Result			**0.014**	**0.80**	

Table 9: Worked example with 5 rounds. Median RTT = 0.80 ms, threshold = 1.5 × 0.80 = 1.20 ms. Round 3 (RTT = 1.22 ms > 1.20 ms) is rejected as an outlier. Final offset is the mean of the 4 kept rounds.

Use Monotonic Clock

Always use

```
time.monotonic()
```

(Python) or

```
CLOCK_MONOTONIC
```

(C). The wall clock

```
time.time()
```

can jump due to NTP adjustments, daylight saving, or manual changes. A jump of even 1 ms during a sync round would corrupt the offset estimate.

```
time.monotonic()
```

is immune to these adjustments and has nanosecond resolution on Linux.

Dual-Camera Integration

Camera Overview

Each gripper unit now carries two cameras:

- **Intel RealSense T265:** Provides 6DoF visual-inertial odometry (VIO) at 200 Hz and fisheye stereo images at 30 Hz. Connected via USB 3.0. Primary role: trajectory tracking and wide-angle observation.
- **USB RGB Camera (wowrobo):** 2MP resolution, 30 fps, connected via USB 2.0 with a 3 m cable. Designed for SO-ARM100/101 gripper integration. Primary role: close-range color observation for policy input.

Camera Specifications

Camera	Resolution	FPS	FOV	Purpose
T265 (fisheye)	848 × 800	30	163°	Wide-angle observation + VIO pose
wowrobo RGB	1920 × 1080	30	78°	Close-range color observation

Table 10: Dual camera specifications per gripper unit.

The T265 uses USB 3.0 bandwidth for stereo fisheye streaming and IMU data. The wowrobo RGB camera uses USB 2.0, which provides sufficient bandwidth for 1080p@30fps MJPEG. Since the RPi 4B has separate USB 3.0 and USB 2.0 controllers, there are no bus contention issues.

Updated Software Architecture

Adding the USB RGB camera and NRF24L01+ radio to the per-gripper software creates a 5-thread architecture:

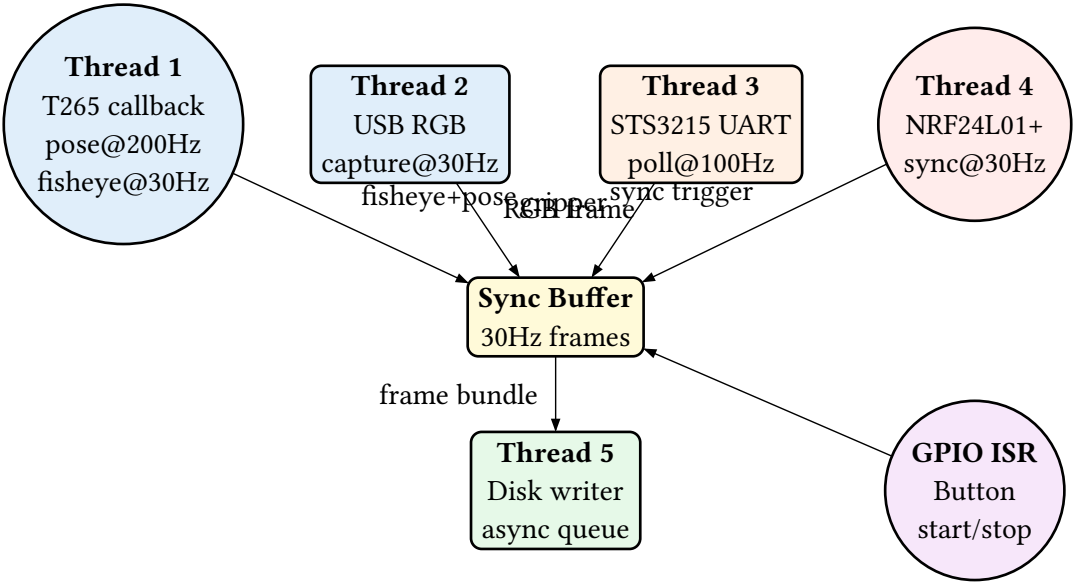

Figure 8: Updated 5-thread software architecture. Thread 4 (NRF) provides the synchronization trigger; Thread 2 (USB RGB) adds color observation.

Frame Synchronization Strategy

The NRF sync trigger from Thread 4 serves as the master clock for frame assembly:

1. **NRF trigger arrives** (Thread 4): Record t_{trigger} using

   ```
   time.monotonic()
   ```

2. **Latch T265 pose:** Select the nearest buffered T265 pose to t_{trigger} (max error ± 2.5 ms at 200 Hz)
3. **Latch T265 fisheye:** Select the nearest fisheye frame (max error ± 16.7 ms at 30 Hz)
4. **Latch USB RGB frame:** Select the nearest RGB frame (max error ± 16.7 ms at 30 Hz)
5. **Latch STS3215 reading:** Select the nearest encoder reading (max error ± 5 ms at 100 Hz)
6. **Bundle and enqueue** for disk writer

Updated LeRobot Dataset Columns

Column	Type	Shape	Description
observation.state	float32	(10,)	Pos (3) + quat (4) + gripper (1) + conf (1) + side (1)
observation.images.fisheye	video	(800, 848, 1)	T265 fisheye frame (MP4)
observation.images.rgb	video	(1080, 1920, 3)	USB RGB camera frame (MP4)
action	float32	(10,)	Target state for next step
timestamp	float64		Seconds from episode start
nrf_rtt_ms	float32		NRF round-trip time (ms)
clock_offset_ms	float32		Estimated clock offset (ms)
episode_index	int64		Episode identifier
frame_index	int64		Frame within episode
index	int64		Global frame index
task_index	int64		Links to tasks.jsonl

Table 11: Updated parquet columns with dual cameras and NRF sync diagnostics.

USB Bandwidth Budget

The RPi 4B has separate USB 3.0 (VL805) and USB 2.0 controllers. The T265 uses USB 3.0 (200 MB/s available). The wowrobo RGB camera uses USB 2.0 (40 MB/s available; 1080p MJPEG@30fps needs 15 MB/s). The NRF24L01+ uses the SPI bus (separate from USB entirely). No bandwidth conflicts exist between any of the three interfaces.

Error Handling and Drift Correction

Packet Loss Scenarios

The NRF24L01+ ESB protocol handles transient packet loss automatically through hardware retransmission (up to 15 retries with configurable delay). However, persistent failures — RF interference, module reset, physical obstruction — require software-level detection and recovery.

Three failure tiers:

1. **Transient loss:** 1–2 packets lost, ESB auto-retry succeeds. Invisible to application. Adds 0.5 ms per retry to RTT.
2. **Burst loss:** 3+ consecutive packets lost despite 15 retries. Software timeout fires (200 ms). Insert empty frame(s) and continue.
3. **Persistent failure:** 3 consecutive software timeouts. Enter RESYNC state, re-run time synchronization, resume recording.

Protocol State Machine

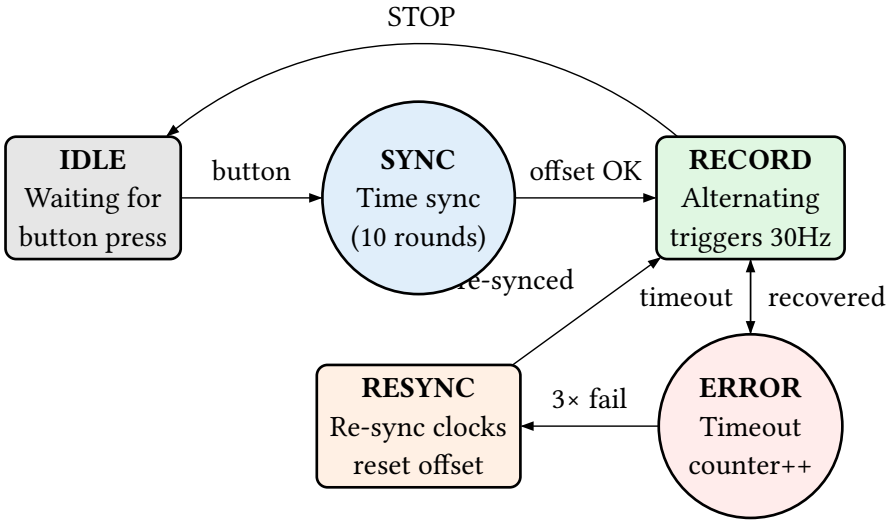

Figure 9: Protocol state machine. ERROR state counts consecutive timeouts; 3 failures trigger RESYNC to re-establish clock offset.

Retry and Timeout Configuration

Parameter	Value	Rationale
Auto Retransmit Delay (ARD)	500 µs	Must exceed ACK payload TX time (164 µs at 2 Mbps)
Auto Retry Count (ARC)	15	Maximum hardware retries before reporting failure
Max HW retry time	8 ms	15 × 500 µs + overhead
Software timeout	200 ms	6× frame period; allows recovery within episode
RESYNC threshold	3 timeouts	3 consecutive SW timeouts → re-sync
Re-sync interval	300 steps	Periodic re-sync to correct drift (every 10 s at 30 Hz)

Table 12: Error handling configuration. Hardware retries handle transient loss; software timeout handles persistent failure.

Clock Drift Correction

Even after initial synchronization, crystal oscillator tolerance causes the clocks to drift apart. Typical quartz crystals have ± 20 ppm tolerance:

$$\Delta\theta(t) = 20 \text{ ppm} \times t$$

At 30 Hz, after 300 steps (10 seconds): $\Delta\theta = 20 \times 10^{-6} \times 10 = 0.2$ ms

This is within our ± 0.4 ms sync error budget. By re-synchronizing every 300 steps, we keep accumulated drift below 0.2 ms. The re-sync is piggybacked on the normal STEP packet by setting the RESYNC flag, so no additional radio transactions are needed.

For longer recording sessions, the re-sync interval can be shortened. The drift formula gives the maximum interval for a target drift bound:

$$t_{\max} = \Delta\frac{\theta_{\max}}{20 \text{ ppm}}$$

For $\Delta\theta_{\max} = 0.1$ ms: $t_{\max} = 0.1 \times \frac{10^{-3}}{20\times10^{-6}} = 5$ s (150 steps).

Packet Loss Recovery

PACKET LOSS RECOVERY — GAP DETECTION

Input: received step number s_{new}, expected step s_{exp}
if $s_{\text{new}} = s_{\text{exp}}$ **then**
    process frame normally
    $s_{\text{exp}} \leftarrow s_{\text{exp}} + 1$ // or $+2$ for alternating
else if $s_{\text{new}} > s_{\text{exp}}$ **then**
    gap $\leftarrow s_{\text{new}} - s_{\text{exp}}$
    for $j \leftarrow 0$ **to** gap -1 **do**
        insert empty frame (copy previous state, mark as interpolated)
    process s_{new} normally
    $s_{\text{exp}} \leftarrow s_{\text{new}} + 1$
    log warning: "gap of {gap} frames at step $\{s_{\text{exp}}\}$"
else // $s_{\text{new}} < s_{\text{exp}}$: duplicate or old packet
    discard (already processed or retransmit artifact)

Figure 10: Gap detection and empty frame insertion. The step number in each packet enables detection of lost frames regardless of timing.

SPI Failure Detection

If the NRF24L01+ STATUS register reads **0x00** or **0xFF**, the SPI bus has failed (loose connection, power glitch, or module fault). Normal STATUS values are 0x0E (idle) or 0x2E (TX success). The software should check STATUS after every

```
radio.write()
```

and

```
radio.available()
```

call. On SPI failure: log error, attempt

```
radio.begin()
```

re-initialization, and if that fails, fall back to unsynchronized recording with a warning flag in the dataset.

Complete Peer Implementation

This chapter presents the complete pseudocode for both peers. Both share the same radio initialization; only the main loop differs between initiator and responder.

Radio Initialization (Shared)

Radio Initialization — Both Peers

```
Input: is_initiator (bool), CE_PIN = 22, CSN_PIN = 0
radio ← RF24(CE_PIN, CSN_PIN)
radio.begin()
radio.setPALevel(RF24_PA_LOW)        // 0 dBm,  10m range
radio.setDataRate(RF24_2MBPS)        // minimum latency
radio.setChannel(108)        // above WiFi band
radio.setRetries(1, 15)        // ARD=500µs, ARC=15
radio.setPayloadSize(32)
radio.enableAckPayload()        // enable ACK payloads
radio.enableDynamicPayloads()
ADDR_A ← b"\xe7\xe7\xe7\xe7\xe7"
ADDR_B ← b"\xc2\xc2\xc2\xc2\xc2"
if is_initiator then
    radio.openWritingPipe(ADDR_A)
    radio.openReadingPipe(1, ADDR_B)
else
    radio.openReadingPipe(1, ADDR_A)
    radio.openWritingPipe(ADDR_B)
radio.flush_tx()
radio.flush_rx()
return radio
```

Figure 11: Shared radio initialization. Channel 108 avoids WiFi. ARD=500µs ensures ACK payload has time to transmit.

Node A — Initiator Main Loop

NODE A — INITIATOR (PTX) MAIN LOOP

Input: radio, offset $\hat{\theta}$, fps $= 30$
step $\leftarrow 0$, timeout_count $\leftarrow 0$
Pre-recording: run time sync ($\rightarrow$ Algorithm 1), get $\hat{\theta}$
loop (until STOP button)
  $t_{\text{start}} \leftarrow$ monotonic()
  capture local sensors (T265 pose, fisheye, RGB, STS3215)
  build STEP packet: step, episode, flags, $t_1 =$ monotonic()
  if step mod $300 = 0$ **then** set RESYNC flag
  success $\leftarrow$ radio.write(packet)
  $t_{\text{done}} \leftarrow$ monotonic()
  if success **and** radio.isAckPayloadAvailable() **then**
    parse ACK payload: remote state, t_2, t_3
    if RESYNC flag was set **then**
      update $\hat{\theta} \leftarrow \frac{(t_2 - t_1) + (t_3 - t_{\text{done}})}{2}$
    store frame: local sensors + remote state + sync metadata
    timeout_count $\leftarrow 0$
  else
    timeout_count $\leftarrow$ timeout_count $+1$
    store frame with empty remote state (mark interpolated)
    if timeout_count ≥ 3 **then**
      enter RESYNC: re-run time sync, reset timeout_count
  step $\leftarrow$ step $+1$
  sleep_until($t_{\text{start}} + \frac{1}{\text{fps}}$)

Figure 12: Node A main loop. Sends STEP packets, reads ACK payloads with remote data. Re-syncs every 300 steps or on persistent failure.

Node B — Responder Main Loop

Node B — Responder (PRX) Main Loop

Input: radio, offset $\hat{\theta}$
radio.startListening()
last_state ← current sensor readings
Pre-load initial ACK payload: pack(last_state, $t_2 = 0$, $t_3 = 0$)
radio.writeAckPayload(1, ack_payload) // pipe 1
loop (until STOP received)
  if radio.available() **then**
    t_2 ← monotonic() // receive timestamp
    read packet from radio
    parse: step, episode, flags, sender t_1
    capture local sensors (T265 pose, fisheye, RGB, STS3215)
    store frame: local sensors + remote t_1 + sync metadata
    if RESYNC flag **then**
      update $\hat{\theta}$ from this round's timestamps
    t_3 ← monotonic() // pre-send timestamp
    build ACK payload: local state, t_2, t_3
    radio.writeAckPayload(1, ack_payload) // for **next** round
  else
    sleep(100 µs) // poll interval

Figure 13: Node B main loop. Stays in PRX mode permanently. Pre-loads ACK payload after each received packet for the next round.

Performance Analysis

Latency Breakdown

Phase	Duration	Cumulative	Notes
SPI TX (32B at 8 MHz)	26 µs	26 µs	32 bytes × 8 bits / 8 MHz
PLL lock	130 µs	156 µs	Crystal stabilization
On-air TX (2 Mbps)	164.5 µs	320.5 µs	(1+5+9/8+32+2) bytes / 2 Mbps
Receiver processing	10 µs	330.5 µs	Address match + CRC check
ACK TX (2 Mbps, 32B)	164.5 µs	495 µs	ACK with payload
SPI RX (ACK payload)	26 µs	521 µs	Read ACK data from FIFO
Software overhead	36 µs	557 µs	Timestamp, pack/unpack
Theoretical RTT		**557 µs**	
Measured RTT		**800 µs**	Linux scheduling adds 243 µs

Table 13: Latency breakdown for a single NRF round trip with 32-byte payload and ACK payload.

Synchronization Error Comparison

Metric	WiFi UDP	NRF24L01+	Improvement
Mean RTT	2.0 ms	0.8 ms	2.5×
RTT std dev	0.5 ms	0.15 ms	3.3×
Sync error (10-round)	0.5 ms	0.2 ms	2.5×
Max sync error	1.5 ms	0.4 ms	3.7×
Drift after 300 steps	0.2 ms	0.2 ms	Same (crystal)
Error bound	Constant	Constant	Both bounded

Table 14: Sync error comparison. NRF achieves 2.5× lower average error. Both methods bound error regardless of recording duration.

Step Timing at 30 Hz

	0–0.8 ms	0.8–1 ms	1–5 ms	5–32 ms	32–33.3 ms
NRF radio	TX+ACK	proc			
Sensors		latch	T265+RGB		
Disk				write	
Idle					sleep
Duty	3%		12%	80%	5%

Table 15: Single-step timing breakdown at 30 Hz (33.3 ms frame period). NRF radio active time is 0.8 ms = 2.4% duty cycle. Majority of time is available for sensor capture and disk writes.

Power Consumption

State	NRF24L01+	WiFi (BCM43455)	Notes
TX active	11.3 mA	300 mA	NRF at 0 dBm; WiFi at 20 dBm
RX active	13.5 mA	100 mA	Listening for packets
Standby-I	26 µA	N/A	NRF between transactions
Power down	900 nA	N/A	Not used during recording
Average at 30 Hz	**50 µA**	**150 mA**	NRF: 2.4% duty cycle
Daily energy (24h)	**4 mWh**	**12 Wh**	NRF: negligible vs RPi 4B (10W)

Table 16: Power comparison. The NRF24L01+ consumes 3000× less power than WiFi. At 30 Hz with 2.4% duty cycle, its contribution to total system power is negligible.

Measured vs Theoretical

The 243 µs gap between theoretical (557 µs) and measured (800 µs) RTT is caused by Linux scheduling latency. The RPi 4B runs a non-realtime kernel; context switches and interrupt handling add 50–200 µs jitter per system call. This jitter is **symmetric** (affects both peers equally on average), so it cancels in the offset calculation. The RTT measurement captures it, but the offset estimate remains accurate to within ± 0.2 ms after multi-round averaging.

References and Hardware Links

QR Code Quick Links

NRF24L01+ Datasheet

pyRF24 Documentation

RPi GPIO Pinout

Intel T265 Camera

LeRobot Framework

UMI Gripper Tutorial

Figure 14: QR codes for key references. Scan with any smartphone camera.

Reference URLs

Resource	URL
NRF24L01+ Datasheet	nordicsemi.com/Products/nRF24-series
pyRF24 Library	nrf24.github.io/RF24/
RPi GPIO Pinout	pinout.xyz
Intel RealSense T265	intelrealsense.com/tracking-camera-t265/
wowrobo USB Camera	wowrobo.com (SO-ARM100/101 compatible)
LeRobot Framework	github.com/huggingface/lerobot
UMI Gripper (Stanford)	umi-gripper.github.io
Diffusion Policy	github.com/real-stanford/diffusion_policy
ACT (Aloha)	github.com/tonyzhaozh/act

Table 17: Complete reference URL list.